



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

D
BACH KHOA

TRÍ TUỆ NHÂN TẠO

N
A
N
G



Khoa Công Nghệ Thông Tin

TS. Nguyễn Văn Hiệu

TRÍ TUỆ NHÂN TẠO

Chương 3: Các phương pháp tìm kiếm có sử dụng thông tin

D
BACH KHOA

N
A
N
G

Nội dung

- Giới thiệu
- Tìm kiếm tham lam tốt nhất
- Tìm kiếm leo đồi
- Tìm kiếm A*
- Bài tập

Giới thiệu

- Các phương pháp tìm kiếm mù kém hiệu quả trong nhiều trường hợp
- Để khắc phục, chương này nghiên cứu sử dụng miền kiến thức:
 - Tìm kiếm đang hướng tới trạng thái đích hay không ?
 - Sử dụng **hàm đánh giá** để đo khoảng cách đến trạng thái đích
- Kỹ thuật tìm kiếm sử dụng hàm đánh giá gọi là tìm kiếm kinh nghiệm
- Các giai đoạn cơ bản của kỹ thuật tìm kiếm kinh nghiệm
 - Tìm biểu diễn thích hợp mô tả các trạng thái và các toán tử.
 - Xây dựng hàm đánh giá,
 - Thiết kế chiến lược chọn trạng thái để phát triển ở mỗi bước

Giới thiệu

- Trong tìm kiếm kinh nghiệm hàm đánh giá có vai trò then chốt
 - Việc xây dựng hàm đánh giá đúng, thì việc tìm kiếm sẽ hiệu quả, không thì ngược lại
 - Việc xây dựng hàm đánh giá tùy thuộc vào vấn đề cần giải
- Ví dụ với bài toán tìm đường đi, thì hàm đánh giá có thể:
 - sử dụng đường chim bay từ tp này đến tp khác
 - sử dụng khoảng cách thực đi giữa các thành phố
 - sử dụng cả khoảng cách thức và trọng số bổ sung trên đường đi
 - Việc xây dựng hàm đánh giá tùy thuộc vào vấn đề cần giải

Giới thiệu

- Các kỹ thuật tìm kiếm kinh được dạy
 - Tìm kiếm ăn tham tốt nhất đầu tiên (Greedy best-first search);
 - Thuật toán leo đồi (Hill-climbing search);
 - Tìm kiếm A^* (A^* search)

Tìm kiếm tham lam tốt nhất đầu tiên



Tìm kiếm tham lam tốt nhất đầu tiên

- Ý tưởng: sử dụng hàm đánh giá kết hợp với tìm kiếm theo chiều rộng
- Hàm đánh giá $h(u)$ ước lượng đến trạng thái kết thúc
- Tìm kiếm tham lam các đỉnh không được phát sinh lần lượt như thuật toán tìm kiếm theo chiều rộng, mà được phát sinh dựa vào hàm đánh giá
- Cài đặt: dùng hàng đợi có sự ưu tiên dựa vào hàm đánh giá.

Tìm kiếm tham lam tốt nhất đầu tiên

- Ý tưởng: sử dụng hàm đánh giá kết hợp với tìm kiếm theo chiều rộng
- Thuật toán tìm kiếm tham lam tốt nhất đầu tiên được mô tả bởi thủ tục:

```
function GREEDY-BEST-FIRST-SEARCH(initialState, goalTest)  
    returns SUCCESS or FAILURE : /* Cost  $f(n) = h(n)$  */
```

```
    frontier = Heap.new(initialState)  
    explored = Set.new()
```

```
    while not frontier.isEmpty():  
        state = frontier.deleteMin()  
        explored.add(state)
```

```
        if goalTest(state):  
            return SUCCESS(state)
```

```
        for neighbor in state.neighbors():  
            if neighbor not in frontier  $\cup$  explored:  
                frontier.insert(neighbor)  
            else if neighbor in frontier:  
                frontier.decreaseKey(neighbor)
```

```
    return FAILURE
```

} khoảng cách
ước lượng
đến đích

Tìm kiếm tham lam tốt nhất đầu tiên

- Hãy làm rõ quá trình biến đổi của danh sách **frontier** ?

Frontier = {A(6)}

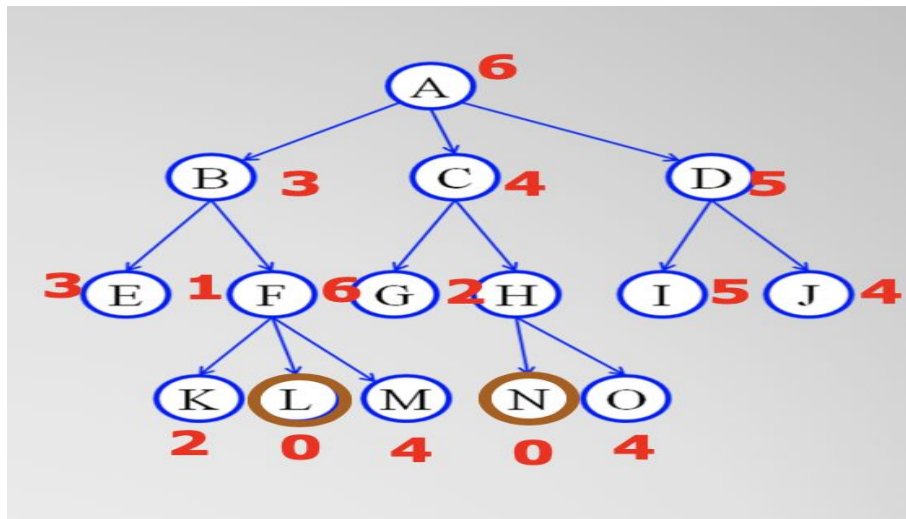
Frontier = {B(3), C(4), D(5)}

Frontier = {F(1), E(3), C(4), D(5)}

Frontier = {L(0), K(2), M(4), E(3), C(4), D(5)}

Frontier =

- Nhận xét: So với Uniform cost search thì Greedy search tiết kiệm về mặt không gian hơn.



```
function GREEDY-BEST-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE : /* Cost  $f(n) = h(n)$  */

    frontier = Heap.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.deleteMin()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier ∪ explored:
                frontier.insert(neighbor)
            else if neighbor in frontier:
                frontier.decreaseKey(neighbor)

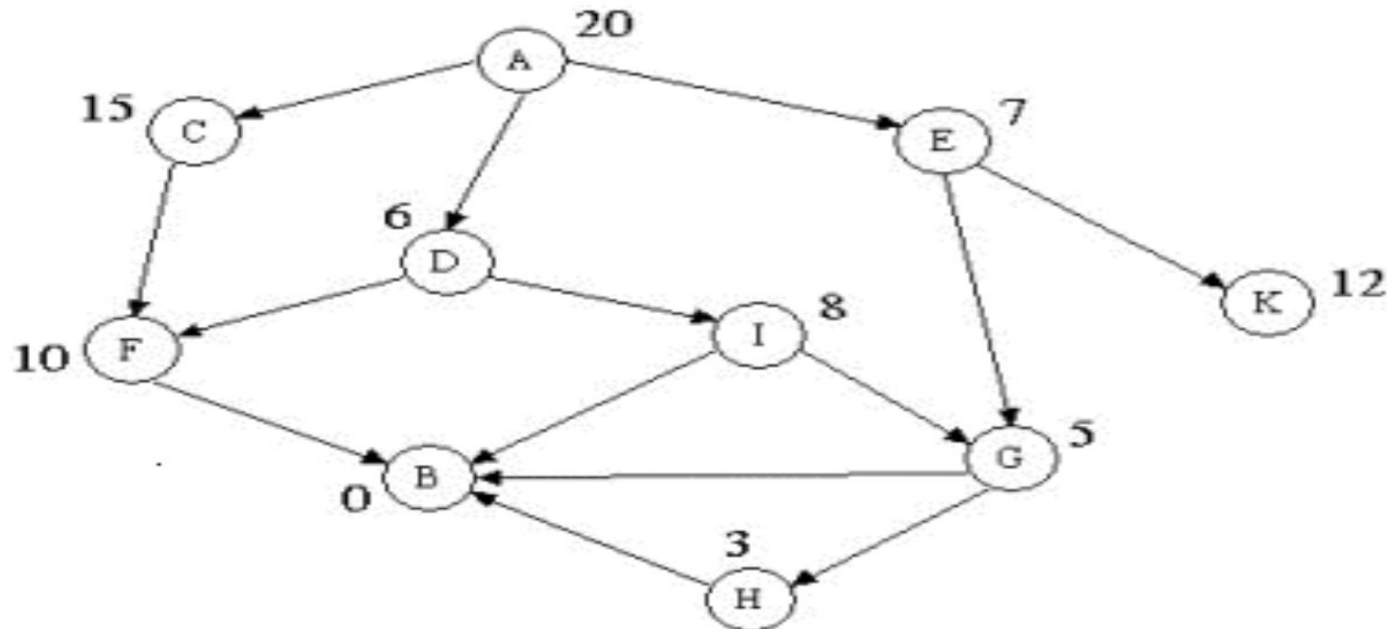
    return FAILURE
```

Tìm kiếm tham lam tốt nhất đầu tiên

- Đánh giá thuật toán tham lam
 - **Tính đủ ?**
 - không (có thể rơi vào vòng lặp luẩn quẩn)
 - **Thời gian ?**:
 - $O(b^m)$
 - Rất xấu nếu $m \gg d$
 - **Không gian?**:
 - $O(b^m)$: lưu trữ tất cả các đỉnh
 - **Tính tối ưu ?**:
 - không

Tìm kiếm tham lam tốt nhất đầu tiên

- Hãy xây dựng quá trình biến đổi **frontier** từ trạng thái A đến trạng thái B



Tìm kiếm leo đồi (Hill-climbing search)

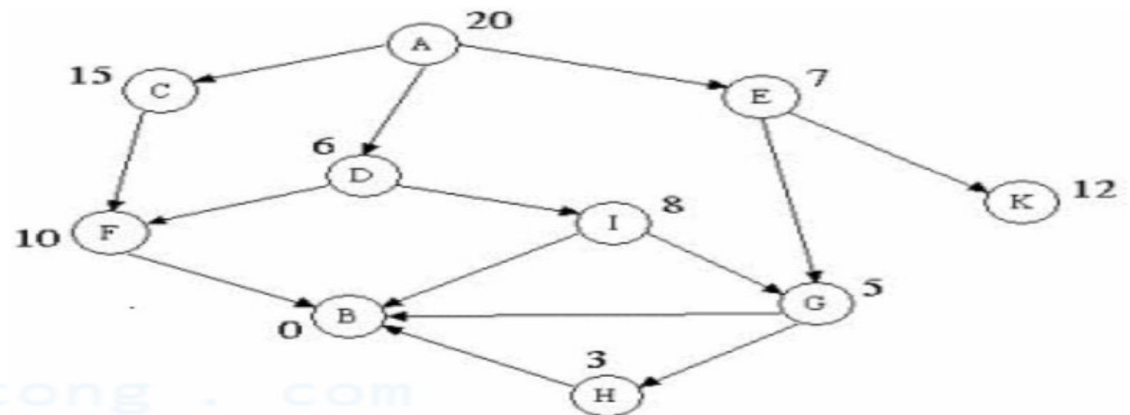


Tìm kiếm leo đồi

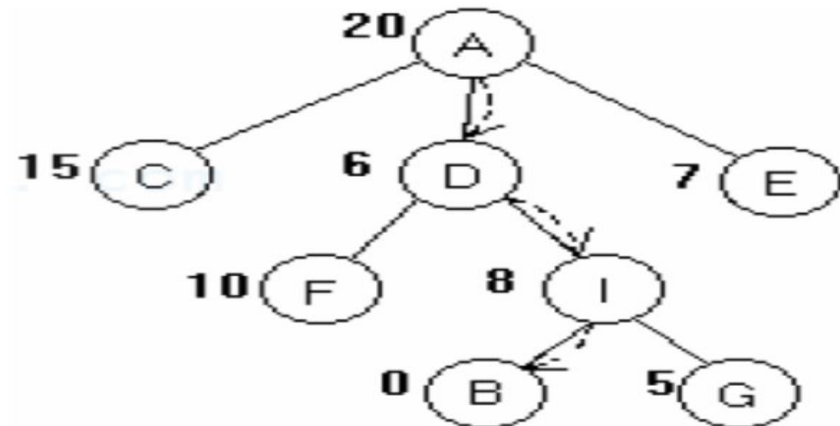
- Ý tưởng: tìm kiếm theo chiều sâu + hàm đánh giá
- Hàm đánh giá $h(u)$ ước lượng đến trạng thái kết thúc
- Tìm kiếm leo đồi khác với kiếm theo chiều sâu khi phát triển đỉnh u , thì chọn trong số các đỉnh con của u đỉnh nào tiềm năng nhất thì phát triển
- Cài đặt: sử dụng ngăn xếp có ưu tiên

Tìm kiếm leo đồi

- Ý tưởng: tìm kiếm theo chiều sâu kết hợp với hàm đánh giá



Xét không gian trạng thái sau



Tìm kiếm A *



Tìm kiếm A *

- Ý tưởng: sử dụng hàm Heuristic chấp nhận được và kỹ thuật tìm kiếm theo chiều rộng
- Hàm Heuristic chấp nhận được
 - $h(u)$ chấp nhận được, nếu $h(u) \leq h^*(u)$, $h^*(u)$ độ dài thực tế từ trạng thái u đến trạng thái đích
 - $h(u)$ không bao giờ đánh giá quá cao so với thực tế
- Đề tăng hiệu quả tìm kiếm
 - $f(u) = g(u) + h(u)$, $g(u)$ độ dài từ trạng thái đầu đến trạng thái u
 - $f(u)$ độ dài lượng giá từ trạng thái đầu tới đích qua trạng thái u

Tìm kiếm A *

- Ví dụ hàm Heuristic chấp nhận được của bài toán 8 số
- $h1(u)$ là số lượng ô bị sai
- $h2(u)$ là tổng số lượng khoảng cách theo Manhattan Metric (số lượng ô từ ô hiện tại đến vị trí mong muốn)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h1(u) = ?$ và $h2(u) = ?$

Tìm kiếm A *

- Ví dụ hàm Heuristic chấp nhận được của bài toán 8 số

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h1(u) = 8$
- $h2(u) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$
- Nếu $h2(u) \geq h1(u)$ với mọi u (cả hai đều chấp nhận được)
 - $h2(u)$ mạnh hơn $h1(u)$ có nghĩa $h2(u)$ tốt hơn $h1(u)$

Tìm kiếm A *

- Ý tưởng: sử dụng hàm Heuristic chấp nhận được và kỹ thuật tìm kiếm theo chiều rộng
- Thuật toán A* được mô tả bởi thủ tục:

```
function A-STAR-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE : /* Cost  $f(n) = g(n) + h(n)$  */

    frontier = Heap.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.deleteMin()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier  $\cup$  explored:
                frontier.insert(neighbor)
            else if neighbor in frontier:
                frontier.decreaseKey(neighbor)

    return FAILURE
```

Tìm kiếm A*

- Hãy làm rõ quá trình biến đổi của danh sách **frontier** ?

frontier = {A(0+6)}

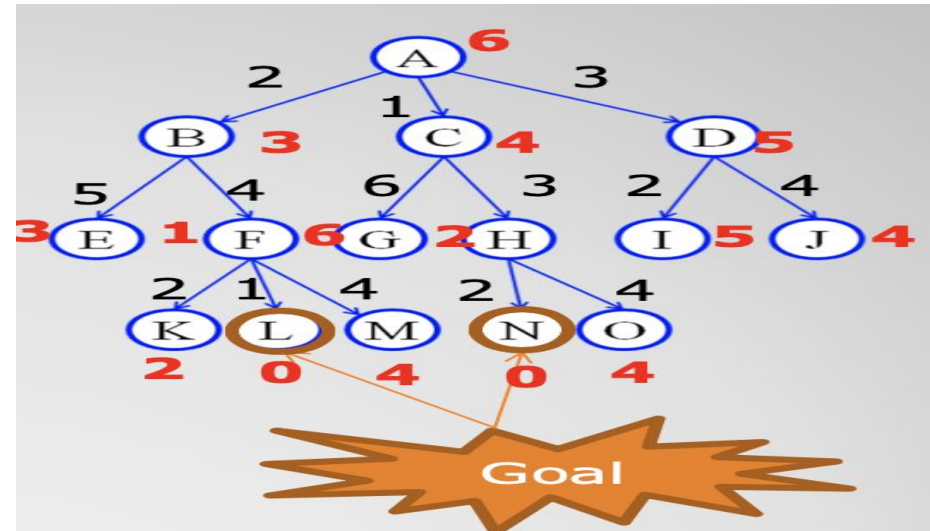
frontier = {B(2+3), C(1+4), D(3+5)}

frontier = {E(7+3),
F(6+1), C(1+4), D(3+5)}

frontier = {E(10),
F(8), G(7+6), H(4+2), D(8)}

frontier = {E(10),
F(8), G(7+6), N(6+0), O(8+4), D(8)}

frontier = ...



```
function A-STAR-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE : /* Cost  $f(n) = g(n) + h(n)$  */

    frontier = Heap.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.deleteMin()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier ∪ explored:
                frontier.insert(neighbor)
            else if neighbor in frontier:
                frontier.decreaseKey(neighbor)

    return FAILURE
```

Bài tập 1

- Xây dựng chương trình cài đặt kỹ thuật tìm kiếm tham lam tốt nhất đầu tiên và kỹ thuật tìm kiếm A*

-

Bài tập 2

- Mỗi sinh viên chọn 01 kỹ thuật tìm kiếm bên dưới để nghiên cứu để hiểu kỹ thuật tìm kiếm đã chọn:
 - Tìm kiếm nhánh và cận (Branch and Bound)
 - Tìm kiếm cục bộ (Local search algorithms)
 - Tìm kiếm mô phỏng luyện kim (Simulated annealing search)
 - Thuật toán gen (Genetic algorithms)